iSky Broker

User Manual & Migration Comparison

Odoo 17 (Market Standards Broker System) → Odoo 20 (iSky_20_Broker)

This manual explains the real-estate brokerage business cycle implemented by the product, and documents what changed going from the original Odoo 17 client build to the generic, configurable Odoo 20 product (iSky_20_Broker). Screenshots were captured from matching Egyptian demo data entered in both systems so the two versions can be compared side by side.

1. The Brokerage Business Cycle

Both versions of the product implement the same real-world sales cycle for a real-estate brokerage. A prospective buyer becomes a **Lead**, is pre-qualified, converted into an **Opportunity** tied to a specific developer project and unit, and then moves through a fixed deal pipeline until the sale is confirmed and a commission is calculated.

1.1 Lead pre-qualification stages

Before a lead becomes a sales opportunity it moves through call-handling stages: **New** → **No Answer / Call Later / Follow Up** → **Qualified** (or **Unqualified**). This pre-qualification layer is unchanged in spirit between v17 and v20, but in v20 the stage list itself is data you can edit from Settings instead of being fixed in code.

1.2 Deal pipeline

Once qualified, the record becomes an Opportunity and advances through the commercial pipeline: **EOI (Expression of Interest)** → **Reservation** → **Contract** → **Confirmed Contract**, with **Cancelled** and **Reopened** as exits/re-entries. Each transition can require sign-off from specific roles (Sales Manager, Financial Manager, Operations) before the next stage is allowed — this is the approval workflow shown later in the Settings screenshots.

1.3 Commission

When a deal reaches Confirmed Contract, the system computes a commission using the job-level sales hierarchy (Salesperson → Team Leader → Sales Manager → Director → Operations) and the project's commission plan/threshold. In v17 these rates and thresholds were hardcoded per client; in v20 they are configurable records and Settings fields, so the same codebase can serve multiple brokerage clients with different commission structures.

2. What Changed Going from v17 to v20

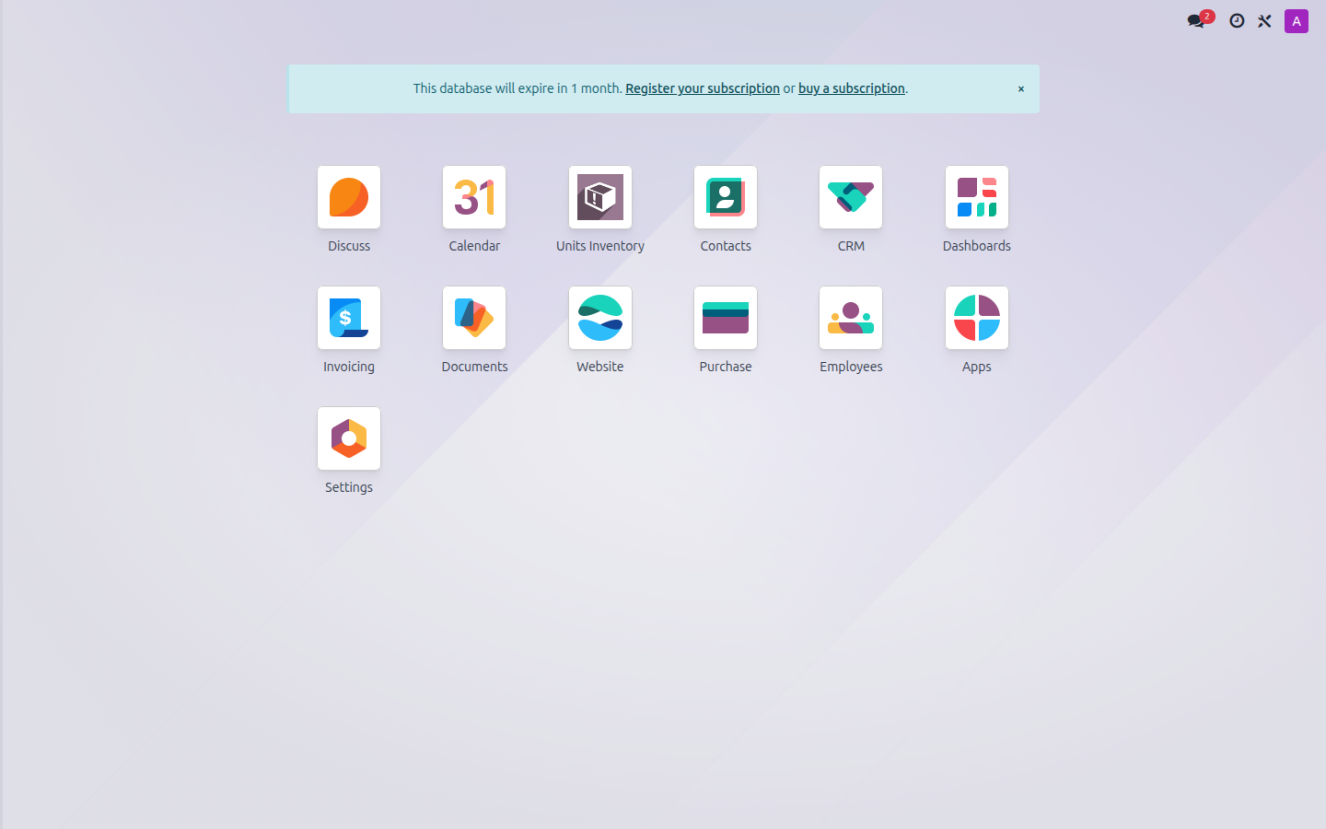
Aspect	Odoo 17 (Market Standards Broker System)	Odoo 20 (iSky_20_Broker)
Module layout	14 modules, client-specific, some hardcoded to one broker	5 modules, generic, installable for any brokerage client
Configurability	Business rules hardcoded in Python (approval chain, thresholds)	Centralized Settings screens (CRM/Commission/Portal blocks) bac
Apps on launcher	Separate "Units Inventory" and "CRM" apps	Consolidated "Broker" app alongside standard CRM
Portal feature control	Always-on portal pages, no toggle	Per-feature portal flags (CRM, Units, Permission Requests) switcha
Multi-company / locale	Single hardcoded company assumption	Full Egypt localization verified: EGP currency, Egypt country/state
Naming convention	isky_*, ks_*, gt_* (mixed, some third-party)	isky_20_broker_* (consistent, one vendor namespace)
API surface	v17 APIs (ir.model.access.csv, hr.contract, groups_id)	v20 APIs (ir.access.csv merged rules, hr.version, models.Constrain

3. Side-by-Side Screenshots

Both systems were seeded with matching Egyptian demo data (New Cairo Towers / Talaat Moustafa Group project family) so the screens below show equivalent records.

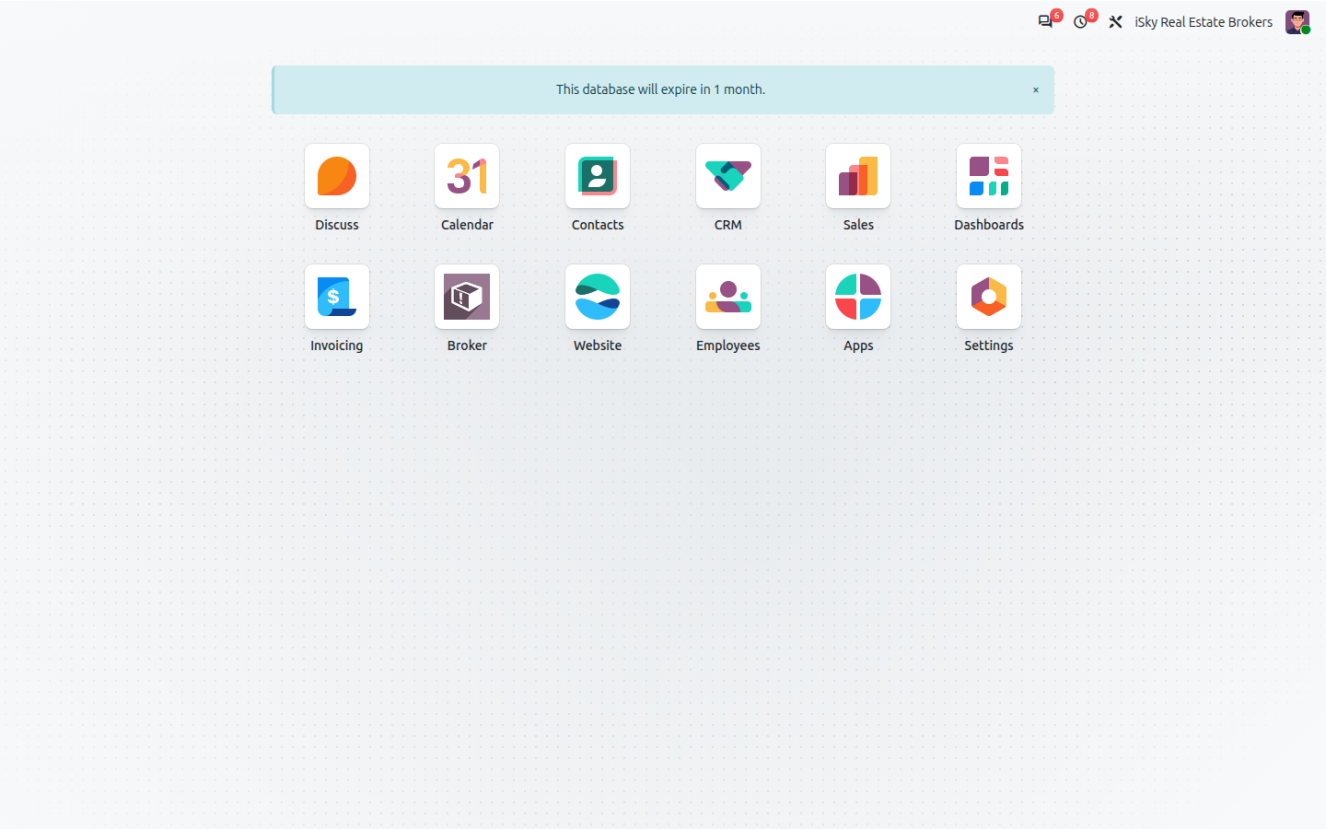
3.1 Application Launcher

ODOO 17



v17: separate "Units Inventory" and "CRM" apps; no broker-specific entry under Settings.

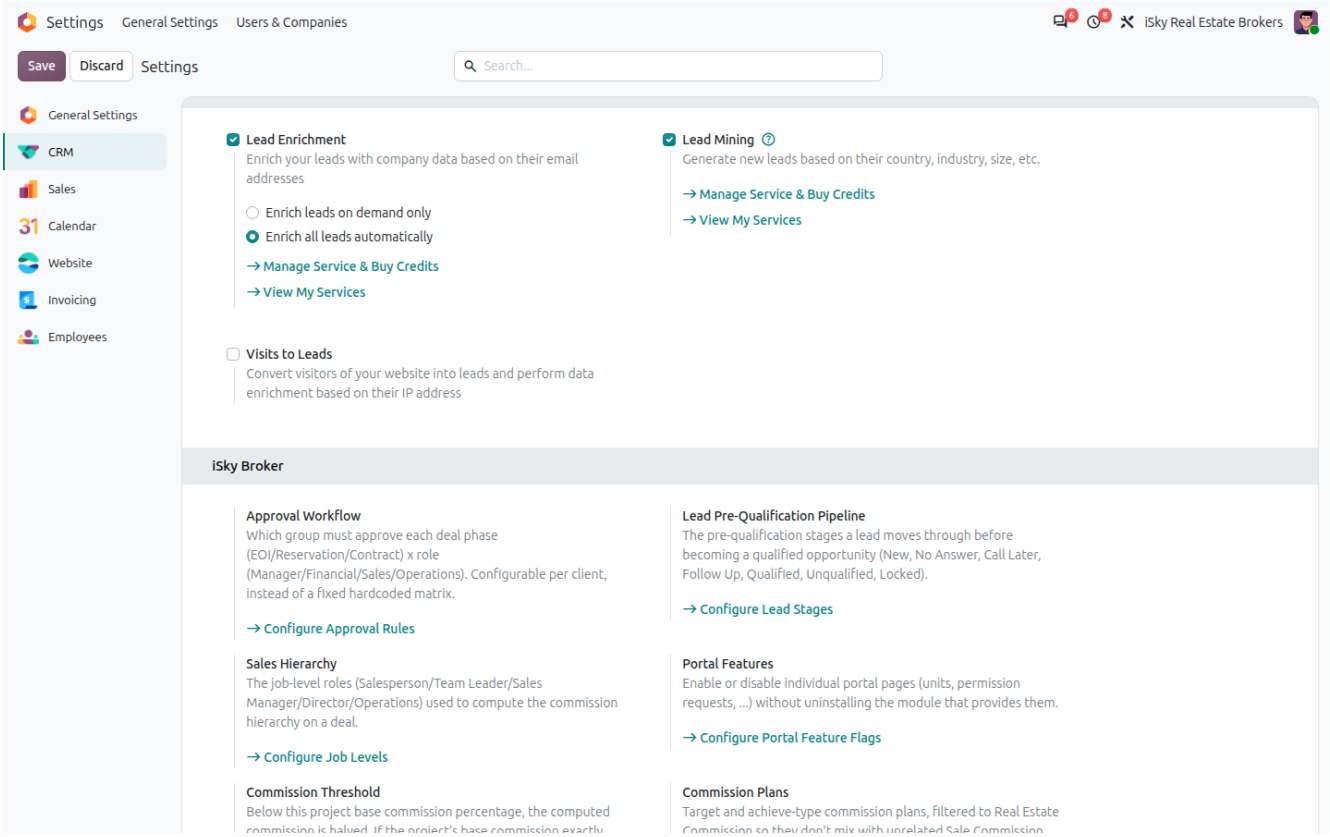
ODOO 20



v20: a single consolidated "Broker" app, multi-company badge ("iSky Real Estate Brokers") in the top bar.

3.2 Configuration (New in v20)

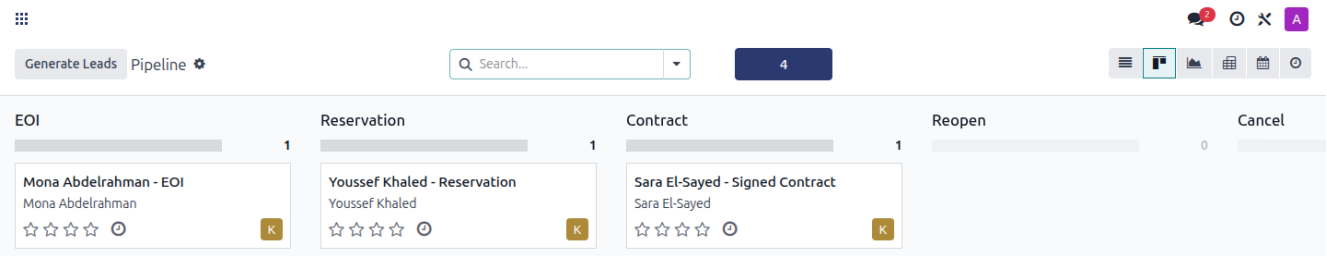
The v17 system had no equivalent screen — every rule below was a hardcoded constant or a Python if/else branch. In v20, the same behavior is exposed under **Settings** → **CRM** → **iSky Broker**: approval workflow, lead pre-qualification stages, sales hierarchy (job levels), portal feature toggles, commission thresholds and plans.



v20 Settings → CRM: the "iSky Broker" configuration block.

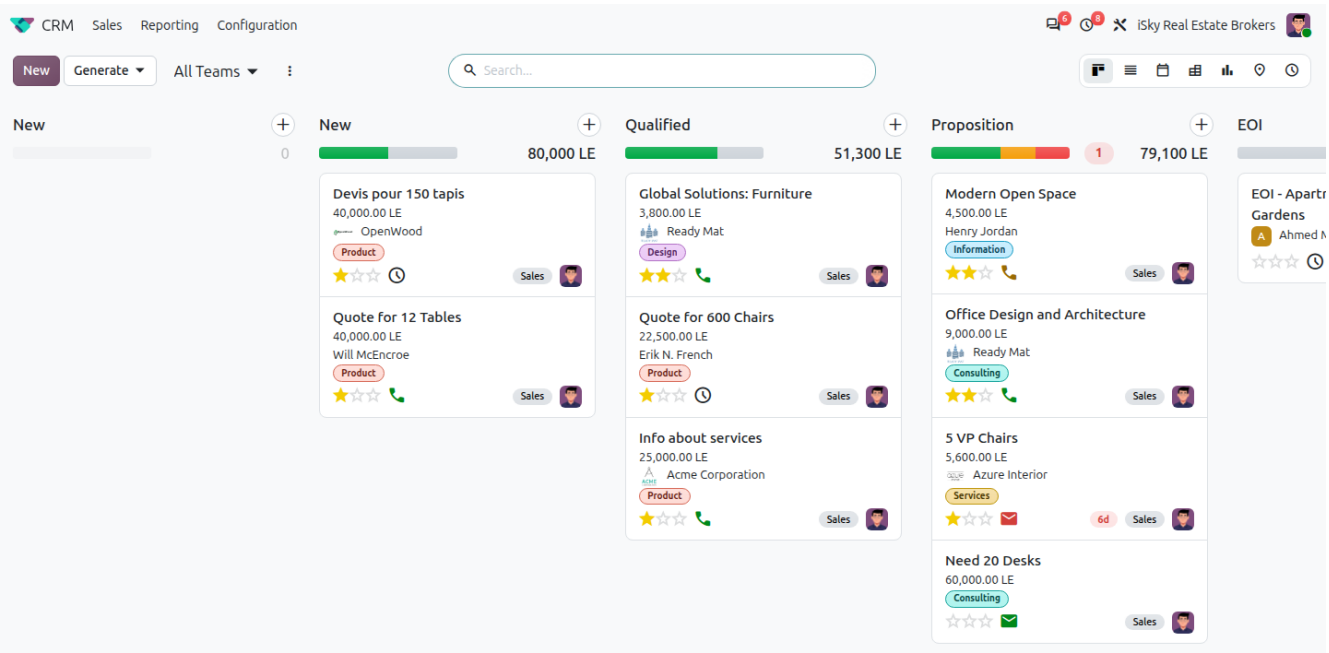
3.3 CRM Pipeline

ODOO 17



v17 Pipeline kanban grouped by deal stage (EOI / Reservation / Contract / Reopen / Cancel), showing the Egyptian demo opportunities.

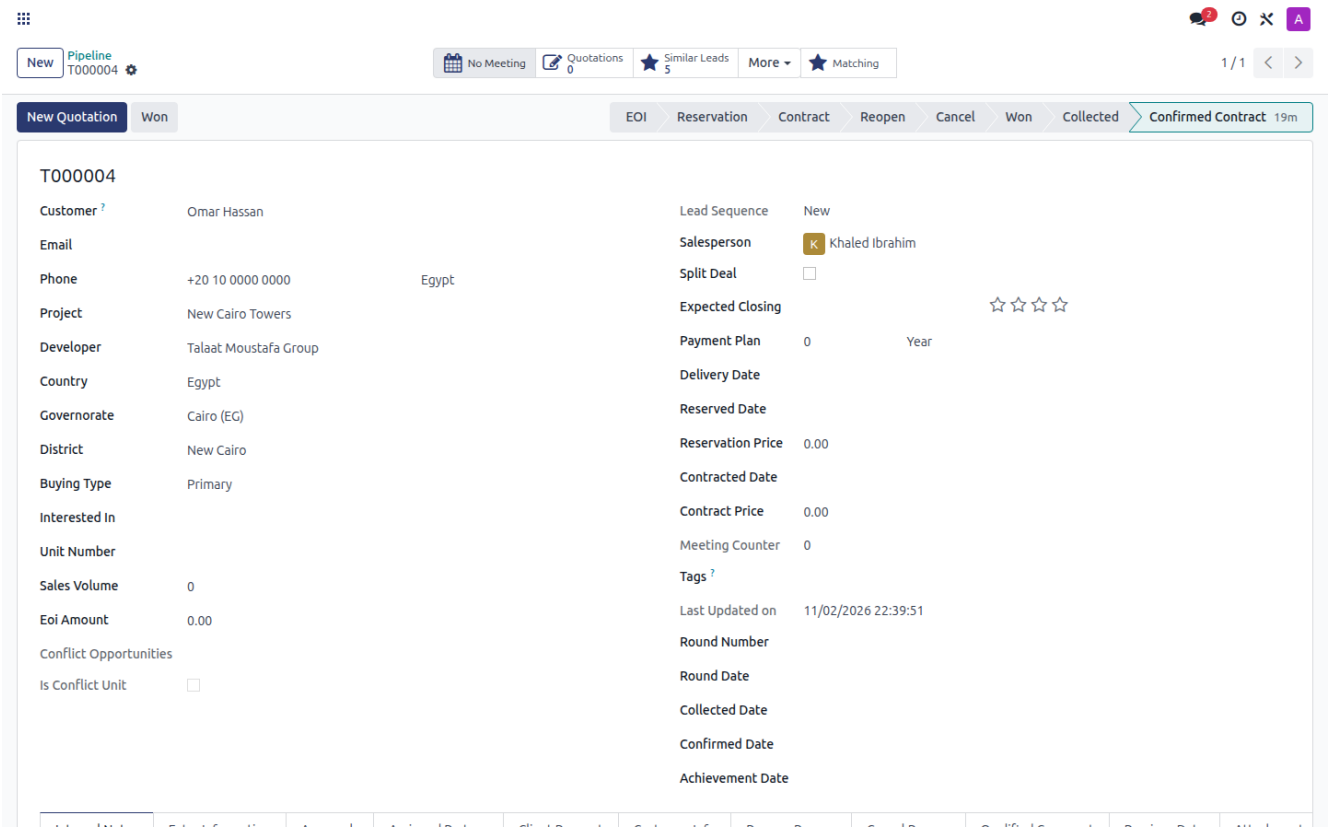
ODOO 20



v20 Pipeline kanban: same stage model, now alongside stock CRM demo leads in the same view (module no longer overrides the whole CRM app).

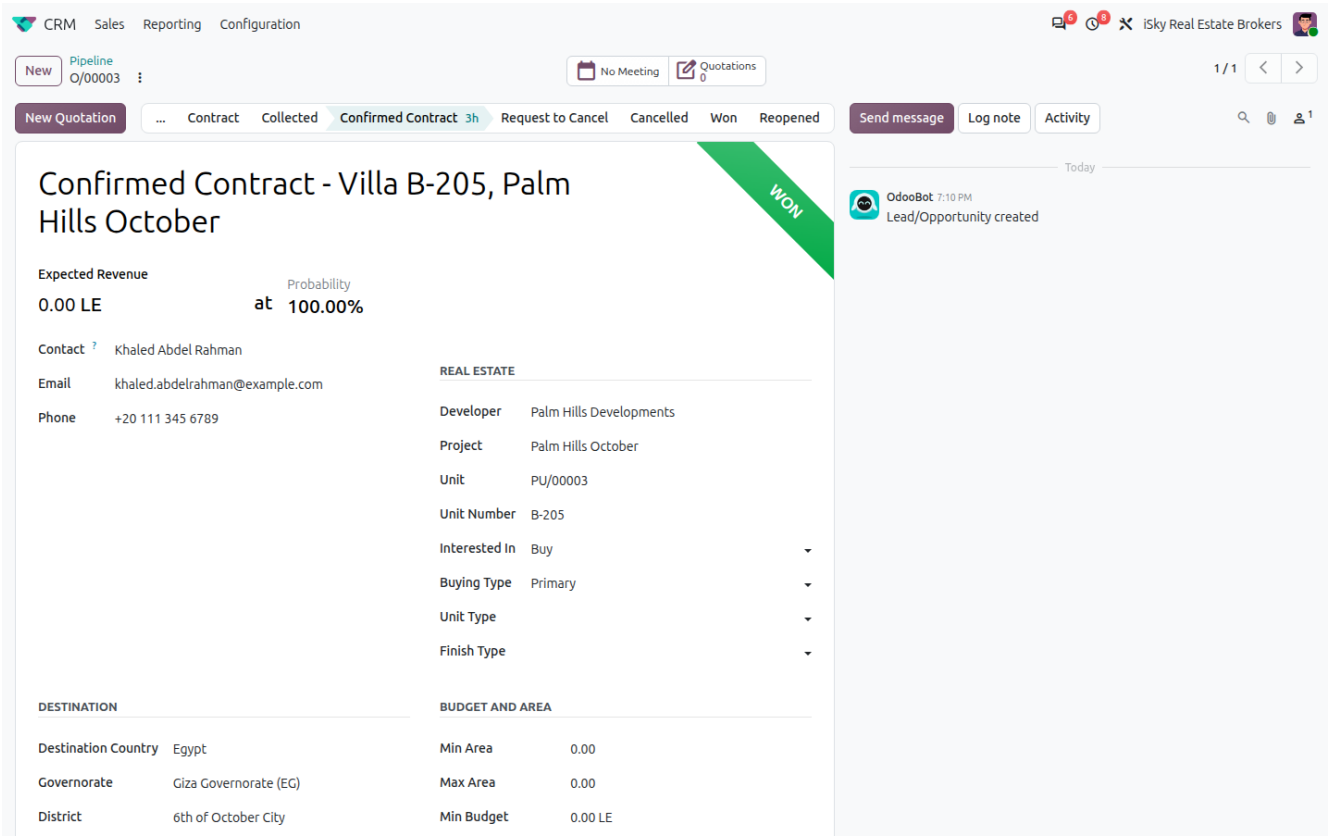
3.4 Opportunity Detail (Confirmed Contract)

ODOO 17



v17 opportunity form: Omar Hassan, New Cairo Towers (Talaat Moustafa Group), stage "Confirmed Contract".

ODOO 20

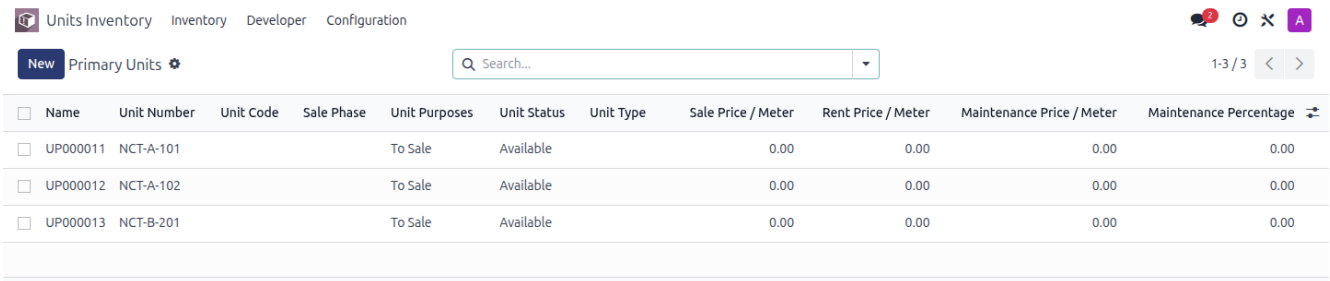


v20 opportunity form: Khaled Abdel Rahman, Palm Hills October (Palm Hills Developments), stage "Confirmed Contract", marked Won.

3.5 Real Estate Inventory (v17)

v17 managed primary-market inventory inside the monolithic broker module. In v20 the same data model lives in the dedicated **isky_20_broker_real_estate** module, reachable from the consolidated Broker app shown in section 3.1.

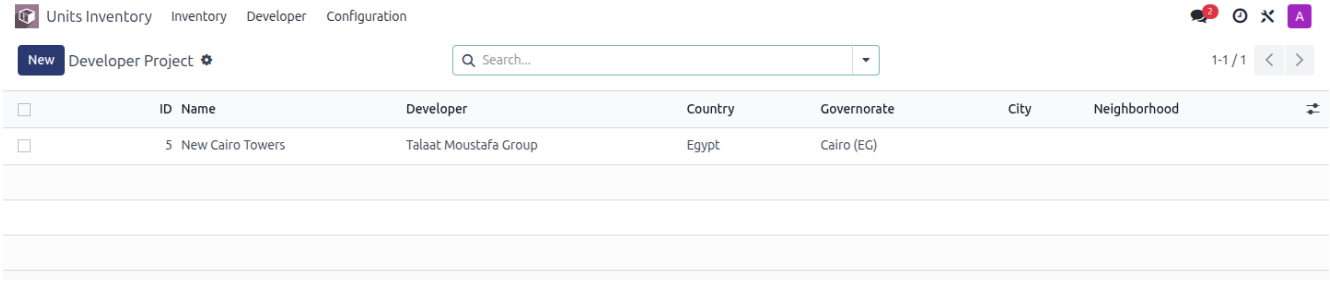
ODOO 17 – PRIMARY UNITS



Name	Unit Number	Unit Code	Sale Phase	Unit Purposes	Unit Status	Unit Type	Sale Price / Meter	Rent Price / Meter	Maintenance Price / Meter	Maintenance Percentage
UP000011	NCT-A-101			To Sale	Available		0.00	0.00	0.00	0.00
UP000012	NCT-A-102			To Sale	Available		0.00	0.00	0.00	0.00
UP000013	NCT-B-201			To Sale	Available		0.00	0.00	0.00	0.00

v17 Primary Units list: 3 Egyptian demo units (NCT-A-101, NCT-A-102, NCT-B-201).

ODOO 17 – DEVELOPER PROJECTS



ID	Name	Developer	Country	Governorate	City	Neighborhood
5	New Cairo Towers	Talaat Moustafa Group	Egypt	Cairo (EG)		

v17 Developer Project list: New Cairo Towers / Talaat Moustafa Group.

4. Summary

The underlying brokerage workflow — lead, pre-qualification, EOI, reservation, contract, confirmed contract, commission — is preserved exactly between v17 and v20. What changed is how much of that workflow is *data you configure* versus *code you fork*: approval chains, pipeline stages, portal visibility, and commission thresholds all moved into Settings and editable records in v20, so the same codebase can be deployed for a new brokerage client without touching Python.

See CLAUDE.md (repository root and per-module) and ARCHITECTURE.md at the repository root for the full module dependency graph, data-flow diagrams, and developer-facing conventions.